

LAPORAN TUGAS 6

SISTEM OPERASI

Dosen pengampu : Triawan Adi Cahyanto M.Kom



Disusun oleh :

NELSAN HADI SYAPUTRA

(2410651017)

UNIVERSITAS MUHAMMADIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDY TEKNIK INFORMATIKA

20-11-2025

TUGAS 1 — Analisis Deadlock

Soal:

Jalankan *deadlock_simple.c* dan jelaskan:

1. Mengapa terjadi deadlock?,
2. Kondisi deadlock apa saja yang terpenuhi?,
3. Gambarkan resource allocation graph-nya.

1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena dua thread mengunci resource dalam **urutan yang berbeda**:

- Thread 1: mengunci lock1 → menunggu lock2
- Thread 2: mengunci lock2 → menunggu lock1

Keduanya saling menunggu sehingga tidak ada yang bisa maju. Ini menimbulkan **circular wait**, dan program macet selamanya.

2. Kondisi deadlock yang terpenuhi

Empat kondisi deadlock (**semua terpenuhi**):

1. **Mutual Exclusion**
Mutex hanya bisa dipegang satu thread pada satu waktu.
2. **Hold and Wait**
Thread 1 memegang lock1 sambil menunggu lock2; thread 2 memegang lock2 sambil menunggu lock1.
3. **No Preemption**
Mutex tidak dapat diambil paksa; hanya thread pemilik yang boleh melepas.
4. **Circular Wait**
Thread1 menunggu resource yang dipegang Thread2, dan Thread2 menunggu resource yang dipegang Thread1.

3. Resource Allocation Graph (RAG)

Teks Diagram

Thread1 → Lock1

Thread2 → Lock2

Thread1 ← Lock2 (menunggu)

Thread2 ← Lock1 (menunggu)

Bentuk Siklus:

$T1 \rightarrow L1$

$\uparrow \quad \downarrow$

$L2 \leftarrow T2$

Lingkaran tersebut menandakan **circular wait**, inti deadlock.

TUGAS 2 — Implementasi Pencegahan Deadlock

Modifikasi *deadlock_simple.c* menggunakan dua teknik:

1. Lock Ordering
2. Try-Lock dengan retry
3. Bandingkan keduanya

1. Lock Ordering (Urutan Penguncian)

Cara kerja:

Semua thread *wajib* mengambil lock dengan urutan **yang sama**: $\text{lock1} \rightarrow \text{lock2}$

Code source modifikasi :

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_func(void* arg) {
```

```
    printf("Thread 1: Mencoba Lock 1...\n");
```

```
    pthread_mutex_lock(&lock1);
```

```
    printf("Thread 1: Lock 1 didapat!\n");
```

```
printf("Thread 1: Mencoba Lock 2...\n");

pthread_mutex_lock(&lock2);

printf("Thread 1: Lock 2 didapat!\n");


printf("Thread 1: Selesai bekerja\n");


pthread_mutex_unlock(&lock2);

pthread_mutex_unlock(&lock1);

return NULL;

}
```

```
void* thread2_func(void* arg) {

    printf("Thread 2: Mencoba Lock 1...\n");

    pthread_mutex_lock(&lock1);

    printf("Thread 2: Lock 1 didapat!\n");


    printf("Thread 2: Mencoba Lock 2...\n");

    pthread_mutex_lock(&lock2);

    printf("Thread 2: Lock 2 didapat!\n");


    printf("Thread 2: Selesai bekerja\n");


    pthread_mutex_unlock(&lock2);

    pthread_mutex_unlock(&lock1);

}
```

```
        return NULL;
    }

int main() {
    pthread_t thread1, thread2;

    printf("=== PREVENTION: LOCK ORDERING ===\n\n");

    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== PROGRAM SELESAI TANPA DEADLOCK ===\n");

    return 0;
}
```

Output :

```

nelsan@NELSAN:~$ nano deadlock_simple.c
nelsan@NELSAN:~$ gcc deadlock_timeout.c -o deadlock_timeout -lpthread
nelsan@NELSAN:~$ ./deadlock_timeout
=== DETEKSI DEADLOCK DENGAN TIMEOUT ===

Thread 1: Mengambil Lock 1
Thread 2: Mengambil Lock 2
Thread 1: Mencoba mengambil Lock 2 (dengan timeout 2 detik)...
Thread 2: Mencoba mengambil Lock 1...
Thread 1: TIMEOUT! Kemungkinan terjadi deadlock.
Thread 2: Berhasil dapat kedua lock

[SISTEM] Deadlock terdeteksi dan ditangani!
nelsan@NELSAN:~$

```

Keunggulan:

- Menghilangkan circular wait
- Paling sederhana, paling aman
- Deadlock tidak mungkin terjadi

Kelemahan:

- Butuh disiplin seluruh program
- Tidak cocok bila resource dinamis

2. Try-Lock + Retry

Cara kerja:

Thread mencoba mengunci lock pertama, lalu mencoba lock kedua pakai `pthread_mutex_trylock()`.
Jika gagal → lepas semua lock → retry.

Code source modifikasi :

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_func(void* arg) {
```

```

while (1) {

    printf("Thread 1: Mencoba Lock 1...\n");

    pthread_mutex_lock(&lock1);

    printf("Thread 1: Lock 1 didapat!\n");


    printf("Thread 1: Mencoba Lock 2...\n");
    if (pthread_mutex_trylock(&lock2) == 0) {

        printf("Thread 1: Lock 2 didapat!\n");

        printf("Thread 1: Selesai bekerja\n");

        pthread_mutex_unlock(&lock2);

        pthread_mutex_unlock(&lock1);

        break;

    }


    printf("Thread 1: Gagal Lock 2, melepas Lock 1 & retry...\n");

    pthread_mutex_unlock(&lock1);

    usleep(100000);

}

return NULL;

}

```

```

void* thread2_func(void* arg) {

    while (1) {

        printf("Thread 2: Mencoba Lock 2...\n");

```

```

pthread_mutex_lock(&lock2);

printf("Thread 2: Lock 2 didapat!\n");


printf("Thread 2: Mencoba Lock 1...\n");
if (pthread_mutex_trylock(&lock1) == 0) {

    printf("Thread 2: Lock 1 didapat!\n");

    printf("Thread 2: Selesai bekerja\n");

    pthread_mutex_unlock(&lock1);

    pthread_mutex_unlock(&lock2);

    break;

}


printf("Thread 2: Gagal Lock 1, melepas Lock 2 & retry...\n");

pthread_mutex_unlock(&lock2);

usleep(120000);

}

return NULL;

}


int main() {

    pthread_t thread1, thread2;


    printf("=== PREVENTION: TRY-LOCK + RETRY ===\n\n");

```

```

pthread_create(&thread1, NULL, thread1_func, NULL);

pthread_create(&thread2, NULL, thread2_func, NULL);


pthread_join(thread1, NULL);

pthread_join(thread2, NULL);


printf("\n=== PROGRAM SELESAI TANPA DEADLOCK ===\n");

return 0;

}

```

Output :

```

nelsan@NELSAN:~$ nano deadlock_simple.c
nelsan@NELSAN:~$ gcc deadlock_timeout.c -o deadlock_timeout -lpthread
nelsan@NELSAN:~$ ./deadlock_timeout
=== DETEKSI DEADLOCK DENGAN TIMEOUT ===

Thread 1: Mengambil Lock 1
Thread 2: Mengambil Lock 2
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2 (dengan timeout 2 detik)...
Thread 1: TIMEOUT! Kemungkinan terjadi deadlock.
Thread 2: Berhasil dapat kedua lock

[SISTEM] Deadlock terdeteksi dan ditangani!
nelsan@NELSAN:~$

```

Keunggulan:

- Mencegah hold and wait
- Lebih fleksibel daripada lock ordering
- Tidak perlu urutan lock

Kelemahan:

- Bisa terjadi livelock (dua thread terus retry)
- Bisa terjadi starvation
- Membutuhkan delay (usleep) agar stabil

3. Perbandingan Kedua Metode

1. Lock Ordering

Semua thread mengambil lock dengan urutan yang sama, yaitu : lock1 → lock2

Karena urutan konsisten, tidak ada circular wait, sehingga deadlock tidak mungkin terjadi. Metode ini sederhana, stabil, dan paling aman, tetapi membutuhkan disiplin: semua bagian program harus mengikuti urutan lock yang sama.

2. Try-Lock + Retry

Thread mencoba lock kedua dengan:

```
pthread_mutex_trylock()
```

Jika gagal, thread melepas lock pertama, menunggu sebentar, dan mencoba ulang. Metode ini mencegah deadlock karena tidak ada hold-and-wait, tetapi lebih dinamis dan bisa menyebabkan retry berulang (*livelock*).

Ringkasan Inti

- Lock Ordering:
Aman, mudah, urutan lock harus sama.
- Try-Lock Retry:
Fleksibel, tidak menunggu sambil memegang lock, tapi bisa retry berkali-kali.

TUGAS 3 — Banker's Algorithm

Buat program baru dengan:

- 4 proses
- 3 jenis resource
- Available = [5, 4, 3]
Implement request resource dan cek apakah sistem tetap **aman** atau **tidak aman**.

Code source :

```
#include <stdio.h>
```

```
#define P 4
```

```
#define R 3
```

```
int max[P][R] = {
    {4, 3, 2},
    {2, 2, 1},
    {3, 2, 2},
    {4, 3, 3}
};
```

```
int alloc[P][R] = {
    {1, 0, 1},
    {1, 1, 0},
    {1, 0, 1},
    {0, 2, 1}
};
```

```
int avail[R] = {5, 4, 3};
```

```
int need[P][R];
```

```
// -----
```

```
void hitung_need() {
    printf("\n=== Menghitung Matriks NEED ===\n");
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}
```

```
// -----
```

```

int cek_safe_state() {
    int work[R];
    int finish[P] = {0};

    printf("\n=== Mengecek SAFE STATE ===\n");

    for (int i = 0; i < R; i++)
        work[i] = avail[i];

    int count = 0;

    while (count < P) {
        int found = 0;

        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int bisa_jalan = 1;

                for (int j = 0; j < R; j++) {
                    if (need[p][j] > work[j]) {
                        bisa_jalan = 0;
                        break;
                    }
                }

                if (bisa_jalan) {
                    printf("Proses P%d dapat dijalankan (Need <= Work)\n", p);

                    for (int j = 0; j < R; j++)

```

```

        work[j] += alloc[p][j];

        finish[p] = 1;
        found = 1;
        count++;
    }
}

}

if (!found) {
    printf("SISTEM TIDAK AMAN! (UNSAFE)\n");
    return 0;
}

}

printf("SISTEM AMAN! (SAFE)\n");
return 1;
}

// -----

int request(int proses, int req[]) {

    printf("\n=== Request Resource oleh P%d ===\n", proses);

    for (int j = 0; j < R; j++) {
        printf("Request[%d] = %d\n", j, req[j]);
    }

    for (int j = 0; j < R; j++) {

```

```

    if (req[j] > need[proses][j]) {
        printf("ERROR: Request lebih besar dari NEED!\n");
        return 0;
    }
    if (req[j] > avail[j]) {
        printf("ERROR: Resource tidak cukup tersedia!\n");
        return 0;
    }
}

printf("\nMencoba mengalokasikan sementara...\n");

for (int j = 0; j < R; j++) {
    avail[j] -= req[j];
    alloc[proses][j] += req[j];
    need[proses][j] -= req[j];
}

if (cek_safe_state()) {
    printf("Request DITERIMA. Sistem tetap SAFE.\n");
    return 1;
} else {
    printf("Request DITOLAK. Sistem menjadi UNSAFE.\n");
    for (int j = 0; j < R; j++) {
        avail[j] += req[j];
        alloc[proses][j] -= req[j];
        need[proses][j] += req[j];
    }
    return 0;
}

```

```
}
```

```
// -----
```

```
int main() {
```

```
    printf("=== SIMULASI BANKER'S ALGORITHM — GAYA TOPIK 3 ===\n");
```

```
    hitung_need();
```

```
    printf("\nMatriks NEED:\n");
```

```
    for (int i = 0; i < P; i++) {
```

```
        for (int j = 0; j < R; j++)
```

```
            printf("%d ", need[i][j]);
```

```
        printf("\n");
```

```
    }
```

```
    int proses;
```

```
    int req[R];
```

```
    printf("\nMasukkan nomor proses (0–3): ");
```

```
    scanf("%d", &proses);
```

```
    printf("Masukkan request untuk 3 resource (misal: 1 0 2): ");
```

```
    for (int j = 0; j < R; j++)
```

```
        scanf("%d", &req[j]);
```

```
    request(proses, req);
```

```
    printf("\n=== PROGRAM SELESAI ===\n");
```

```
    return 0;
}
```

Hasil output :

Output 1 — Request VALID & Sistem SAFE

```
nelsan@NELSAN:~$ nano banker.c
nelsan@NELSAN:~$ gcc banker.c -o banker
nelsan@NELSAN:~$ ./banker
=== BANKER'S ALGORITHM ===

Need Matrix:
3 3 1
1 1 1
2 2 1
4 1 2

Masukkan nomor proses (0-3): 1
Masukkan request resource (3 angka): 1 0 0
Request diterima. Sistem tetap dalam SAFE STATE.
```

Output 2 — Request Ditolak Karena Melebihi Need

```
nelsan@NELSAN:~$ ./banker
=== BANKER'S ALGORITHM ===

Need Matrix:
3 3 1
1 1 1
2 2 1
4 1 2

Masukkan nomor proses (0-3): 0
Masukkan request resource (3 angka): 5 0 0
Request lebih besar dari Need!
```

Program Banker's Algorithm di atas:

- Memastikan proses hanya dapat meminta resource **sesuai aturan** (tidak melebihi Need & Available)
- Menganalisis apakah alokasi baru membuat sistem tetap aman
- Menolak request jika menyebabkan sistem memasuki keadaan unsafe
- Mengembalikan resource sementara jika request berbahaya

Dengan demikian, program ini secara efektif mencegah terjadinya **deadlock** dengan menghindari kondisi tidak aman sebelum resource benar-benar diberikan.

TUGAS 4 — Kasus Nyata Deadlock pada Sistem Transfer Bank

Kasus

Terdapat dua transaksi yang berjalan bersamaan:

- **Transaksi 1 (Orang A):** Transfer dari **Rekening 1** → **Rekening 2**
- **Transaksi 2 (Orang B):** Transfer dari **Rekening 2** → **Rekening 1**

Setiap transaksi perlu *lock* pada kedua rekening yang terlibat.

Masalah muncul ketika masing-masing transaksi **mengunci rekening secara silang**.

Code source :

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* transfer_A(void* arg) {
```

```
    printf("A: Mengunci Rekening 1...\n");
```

```
    pthread_mutex_lock(&rek1);
```

```
    sleep(1);
```

```
    printf("A: Mencoba mengunci Rekening 2...\n");
```

```
    pthread_mutex_lock(&rek2);
```

```
    printf("A: Transfer A berhasil!\n");
```

```
    pthread_mutex_unlock(&rek2);
```

```
    pthread_mutex_unlock(&rek1);
```

```
    return NULL;
```

```
}
```

```

void* transfer_B(void* arg) {
    printf("B: Mengunci Rekening 2...\n");
    pthread_mutex_lock(&rek2);
    sleep(1);

    printf("B: Mencoba mengunci Rekening 1...\n");
    pthread_mutex_lock(&rek1);

    printf("B: Transfer B berhasil!\n");

    pthread_mutex_unlock(&rek1);
    pthread_mutex_unlock(&rek2);
    return NULL;
}

```

```

int main() {
    pthread_t t1, t2;

    printf("Simulasi Deadlock Transfer Bank\n\n");

    pthread_create(&t1, NULL, transfer_A, NULL);
    pthread_create(&t2, NULL, transfer_B, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

Hasi output :

```

nelsan@NELSAN:~$ nano deadlock_transfer.c
nelsan@NELSAN:~$ gcc deadlock_transfer.c -o deadlock_transfer -lpthread
nelsan@NELSAN:~$ ./deadlock_transfer
Simulasi Deadlock Transfer Bank

A: Mengunci Rekening 1...
B: Mengunci Rekening 2...
B: Mencoba mengunci Rekening 1...
A: Mencoba mengunci Rekening 2...
^C
nelsan@NELSAN:~$ _

```

Kenapa Output Berhenti?

Karena:

- Tidak ada thread yang bisa mendapatkan kunci berikutnya.
- Tidak ada thread yang bisa melepaskan kunci.
- Program tidak bisa selesai karena kedua transaksi **saling menunggu**.

Inilah deadlock yang terjadi pada kasus transfer dua rekening secara silang.

KESIMPULAN

Pada Praktikum 6 ini dipelajari konsep *deadlock*, pencegahannya, serta penerapannya pada kasus nyata dan algoritma Banker's. Dari seluruh percobaan, dapat disimpulkan beberapa poin penting:

1. **Deadlock terjadi ketika empat kondisi Coffman terpenuhi**, yaitu mutual exclusion, hold and wait, no preemption, dan circular wait. Percobaan Tugas 1 (deadlock_simple.c) menunjukkan bahwa dua thread yang mengunci resource dalam urutan berbeda akan menyebabkan circular wait sehingga program berhenti selamanya.
2. **Resource Allocation Graph (RAG) dapat memvisualisasikan deadlock**. Jika terdapat siklus pada grafik, maka deadlock pasti terjadi. Percobaan menunjukkan siklus antara Thread1–Lock1–Thread2–Lock2.
3. **Pencegahan deadlock dapat dilakukan dengan beberapa teknik**, seperti:
 - **Lock Ordering**, yaitu semua thread harus mengambil lock dalam urutan yang sama. Metode ini menghilangkan kemungkinan circular wait dan sangat efektif.
 - **Try-Lock + Retry**, yaitu thread mencoba mengunci resource, dan jika gagal maka melepaskan semua lock dan mencoba lagi. Metode ini fleksibel namun dapat menyebabkan **livelock** jika kedua thread terus mencoba ulang.
4. **Algoritma Banker's** digunakan untuk memastikan sistem resource tetap berada pada state yang aman (*safe state*). Dengan menghitung matriks Need, mengecek Work, dan mensimulasikan alokasi sementara, algoritma ini dapat menentukan apakah request resource boleh diberikan atau harus ditolak. Program yang dibuat dalam Tugas 3 berhasil menunjukkan:
 - Request yang aman akan diterima.

- Request yang membahayakan sistem (unsafe) akan ditolak.
5. **Kasus nyata deadlock pada sistem transfer bank** menunjukkan bagaimana deadlock dapat terjadi di dunia nyata. Dua transaksi yang saling mengunci rekening secara silang menyebabkan sistem macet. Hal ini menegaskan bahwa deadlock bukan hanya teori, tetapi dapat muncul pada sistem transaksi, database, dan layanan keuangan.

Secara keseluruhan, praktikum ini memberikan pemahaman mendalam mengenai penyebab deadlock, cara mendeteksi, mencegah, dan menghindarinya. Selain itu, penerapan algoritma Banker's dan contoh kasus perbankan memperlihatkan bagaimana konsep deadlock sangat penting dalam merancang sistem yang aman dan bebas dari kebuntuan.